

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Construcción y evolución de software

Diagramas UML

Nombres:

Víctor Manuel González Ruiz.

García Ugarte Javier.

Laura Sarai Mares Pacheco

Matricula:

17-011-0529

15-011-0163

23-011-1462

Profesor: Emmanuel Vite Sevilla.

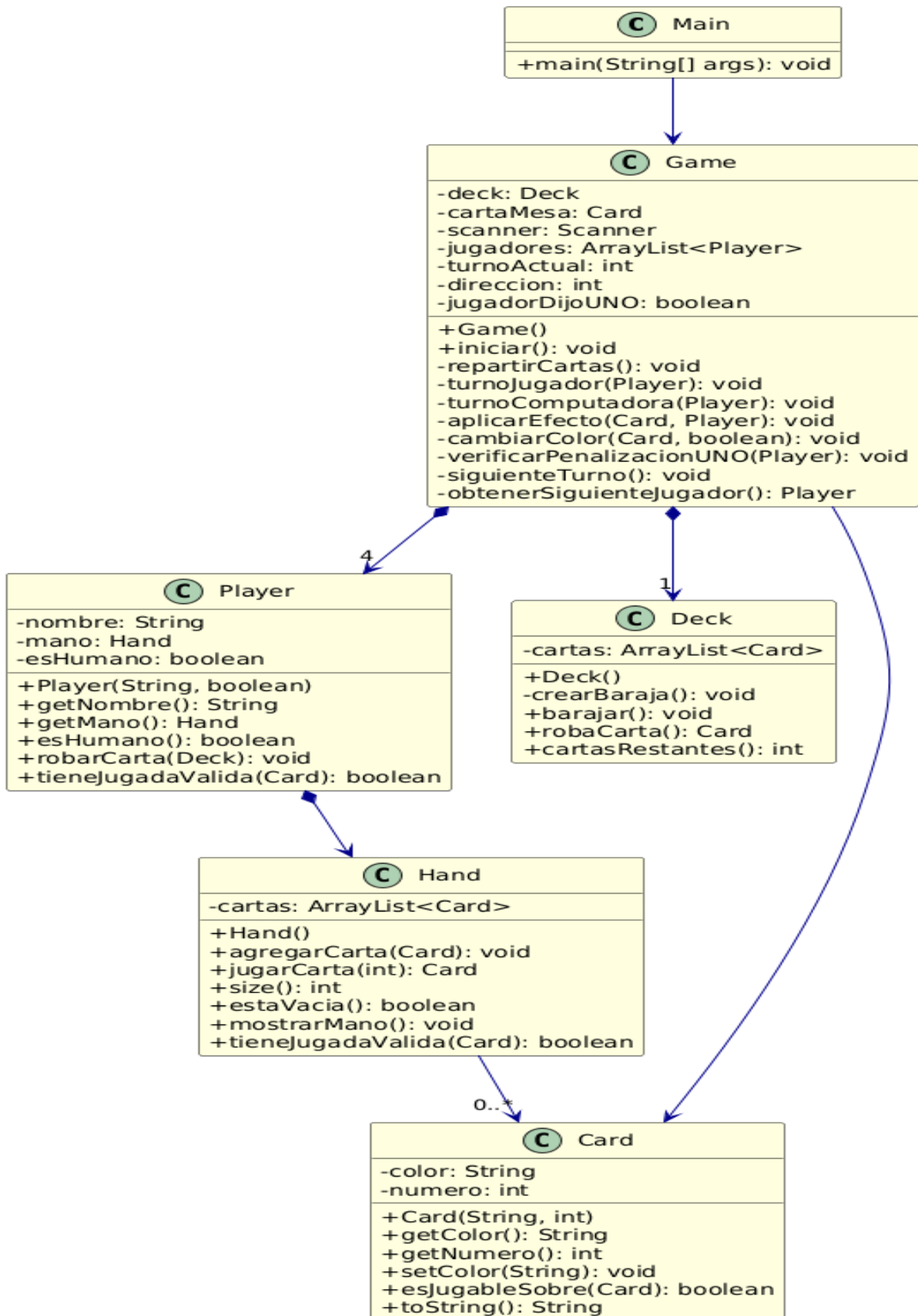
DOCUMENTACIÓN COMPLETA - JUEGO UNO VERSIÓN 3

Introducción al Diagrama UML

El diagrama UML presentado representa la arquitectura de la **Versión 3 del juego de UNO**, la cual ha sido refactorizada para soportar de 2 a 4 jugadores (1 humano + 3 bots), implementar reglas faltantes como +2, salto, reversa, comodín y roba 4, además de mejorar el manejo de turnos y penalizaciones UNO.

Las clases principales son:

- **Main:** Punto de entrada del programa.
- **Game:** Controlador central que gestiona la lógica del juego, turnos, dirección, penalizaciones y efectos especiales.
- **Deck:** Baraja con 108 cartas, métodos para crear, barajar y robar.
- **Card:** Representa una carta con color y número, valida jugabilidad.
- **Player:** Nuevo jugador con nombre, mano y tipo (humano o bot).
- **Hand:** Conjunto de cartas del jugador con métodos para jugar, agregar y mostrar.



DOCUMENTO: ¿QUÉ SE MEJORÓ Y POR QUÉ?

Aspecto mejorado	Versión anterior	Versión 3 actual	Por qué
Número de jugadores	Solo 2 (humano vs 1 bot)	2-4 jugadores (1 humano + 3 bots)	Mayor realismo y dinamismo
Estructura de jugadores	Variables separadas (jugador, computadora)	ArrayList<Player> genérico	Escalabilidad, fácil añadir más jugadores
Dirección de turnos	Fija (solo avance)	Variable con direccion (+1/-1)	Implementar carta REVERSA correctamente
Efecto SALTO	Saltaba solo un turno fijo	Salta usando siguienteTurno()	Juego más fiel a las reglas reales
Efecto +2 y ROBA 4	Solo 2 cartas, sin perder turno	Roba + pierde turno del siguiente	Reglas oficiales de UNO
Penalización UNO en bots	Solo humano penalizado	Bots tienen 70% de decir UNO, 30% penalización	Mayor dificultad y realismo
Método robarCarta()	Directo en Game	Dentro de Player	Encapsulación, menos acoplamiento
Manejo de turnos	Turno manual	siguienteTurno() automático	Reducción de código repetitivo

Validación de jugada	Solo en Game	También en Player y Hand	Separación de responsabilidades
Excepciones implícitas	Sin manejo	Validación de entrada con hasNextInt()	Evita crashes por entrada incorrecta

¿Qué se mejoró con respecto a la segunda versión y por qué?

En la segunda versión del proyecto, el sistema experimentó una evolución importante en términos de estructura, lógica del juego y realismo en la implementación de las reglas. Mientras que en la primera mejora ya se había logrado un juego funcional con múltiples jugadores, esta segunda versión se enfocó en refinar el comportamiento del sistema y hacerlo más fiel al funcionamiento real del juego UNO.

Una de las principales mejoras fue la optimización en la gestión de turnos. En versiones anteriores, el control del turno siguiente se realizaba de forma directa dentro de los efectos de las cartas, lo que generaba cierta redundancia y posibles errores en la lógica. En esta versión, se implementó un método específico para obtener al siguiente jugador (`obtenerSiguienteJugador`), lo que permitió separar la lógica de consulta del turno de la modificación real del mismo. Esto mejora la claridad del código y evita efectos secundarios no deseados.

Otra mejora clave fue la corrección y refinamiento de las cartas especiales, especialmente las cartas de tipo +2 y +4. En versiones anteriores, la lógica podía provocar comportamientos inconsistentes al momento de avanzar turnos o seleccionar al jugador afectado. En la segunda versión, se estableció de manera más clara quién es el jugador objetivo, aplicando correctamente la penalización y asegurando que pierda su turno. Esto hace que la mecánica sea más justa y coherente con las reglas del juego.

Asimismo, se mejoró la modularidad del código. Métodos como `aplicarEfecto` fueron reorganizados para manejar de manera más limpia cada tipo de carta, reduciendo la duplicación de código y facilitando futuras modificaciones. Esta mejora responde a la necesidad de mantener un código más ordenado y fácil de mantener conforme el proyecto crece en complejidad.

También se refinó la lógica de los bots, manteniendo una inteligencia artificial básica pero más integrada al flujo general del juego. Ahora los bots utilizan la misma estructura de turnos que el jugador humano, lo que unifica el comportamiento del sistema y evita inconsistencias en la ejecución.

Una mejora adicional importante fue la implementación más completa de la regla de "UNO". En esta versión, no solo se mantiene la penalización para el jugador humano, sino que también se introduce un comportamiento probabilístico para los bots, simulando la posibilidad de que olviden decir UNO. Esto añade un elemento más realista y dinámico al juego, haciendo que los oponentes se comporten de manera menos predecible.

Finalmente, se logró una mayor estabilidad general del sistema. Se corrigieron errores relacionados con variables mal definidas, control de flujo y estructuras condicionales, lo que permitió que el juego funcione de manera más fluida y sin fallos durante su ejecución.

En conclusión, la segunda versión no solo mejora la funcionalidad del sistema, sino que también optimiza su estructura interna, corrige inconsistencias y añade elementos que enriquecen la experiencia del usuario. Estas mejoras fueron necesarias para transformar el proyecto en un sistema más robusto, mantenible y cercano a una implementación real del juego UNO.